

For each of the five C++ classes that your group will be developing, you should include in your writeup:

- 1) A **class description**, including what its goals are, and what its high-level functionality should look like. This does not need to perfectly match the description you were given, but it should be in the same general spirit unless you confirm any changes with the instructors ahead of time.
- 2) A list of **similar classes** in the standard library that you should be familiar with or use to inform the functionality you will be developing.
- 3) A list of **key functions** that you plan to implement. This does not need to be an exhaustive list, but it should give a strong indication of how the class should be used.
- 4) A set of **error conditions** that you will be responsive to. For each, indicate if its source was (1) *programmer error*, such as invalid arguments to a function, which should be caught by an assert, (2) a potentially *recoverable error*, such as a resource limitation, which should trigger an exception, or (3) a *user error*, such as invalid input, which needs to simply return a special condition for the programmer to respond to (note: these can usually be handled with exceptions as well if you prefer.)
- 5) Any **expected challenges** that you will be facing, an especially any extra topics you need to learn about. This information will help me make sure to cover topics early in the course or else provide your group with extra guidance on the project.
- 6) A list of any **other group's C++ classes** that you think you may want to coordinate with (e.g., to have a compatible interface).

You can make some refinements to these specifications later, but will need to consult with the instructors before making wholesale changes.

In addition to the write-ups for your five C++ classes, you should also include a paragraph about your **vision for the main module** that you will develop. This description can be fairly high-level at this point, but will provide a concrete starting point to work with the other groups in your company on the bigger picture.

Datum:

Class description and goals:

The Datum class functions as a single value that can be stored and used as more than one type, mainly as a string or a double but I would like to also add support for boolean values. The main goal is to make it simple to work with data values without always needing to know the type. Under the hood Datum stores its value as an `std::variant` so it can always know the original type

Similar classes:

1. `std::variant`

2. `std::string`
3. `std::to_string`
4. `std::stod`
5. `Std::optional`

Key functions:

Key functions for Datum include `AsString()` and `AsDouble()` which allow the value to be read in the type the programmer needs. If a value is stored as a number and is called with `AsString()`, it is automatically converted. If the value is stored as a string and a number is requested, the class attempts to convert it. This class will also support all basic operators and arithmetic as if they were the original types, if an operation doesn't make sense it safely handles the situation without crashing or throwing an exception.

Error conditions:

User error (2) conditions for this class include illegal arithmetic such as division by zero and attempting to perform arithmetic on illegal objects (strings). In each of these cases the operations will return NaN (not a number) and the programmer can check for this using `std::isnan()` and respond accordingly. Additionally if the programmer attempts to call `AsDouble()` on a string that cannot be converted to a number, the class will also return NaN to allow the programmer to respond. Finally if comparison is requested for two values that cannot be converted to a common type, the class will handle this in the same way.

Expected challenges:

The main expected challenge I have for Datum is handling errors without using exceptions as we are avoiding them for performance reasons. Instead, Datum will utilize return values and standard library functions such as `std::isnan()`. Another challenge is handling edge cases, including floating point precision errors, infinity, and NaN. For this class I will have to familiarize myself with the `std::variant` in order to use it correctly and safely. Finally, operator overloading could be tricky when deciding when to treat values as numbers, strings, or if the types do not match at all.

Other groups useful C++ classes:

This class will obviously coordinate with the `DataGrid` class which is a two dimensional array of Datum values. Additionally `Serializer` needs a way to convert values to and from strings, which Datum already does. `DataMap` from Group 7 stores named values with arbitrary types, and Datum can serve as a flexible alternative if the types are not necessarily known at compile time. This same concept applies to `ExpressionParser` from Group 5 which may also not know the types ahead of time.

DataGrid: The main data structure behind the application. Agents will regularly access information about the world, player, and more through `DataGrid` objects. Data will be compactly stored in Datum objects and can be requested in a multitude of forms

including, by rows, columns, or automatically converted to relevant data types using a Serializer.

Similar classes: `std::vector`

Functions:

- **DataGrid(int rows, int cols):** Constructor that allocates memory with the specified capacity.
- **Void Insert(int row, int col, T element):** Converts a new element into a Datum object and places it in the specified location in the grid.
- **Void Insert(T element):** Adds an element to the closest available empty space after converting it to a Datum object.
- **Void Append(std::vector<T> row):** Adds a row to the bottom of the DataGrid.
- **std::vector<Datum> Row(int r):** returns an entire row of the grid.
- **std::vector<Datum> Column(int c):** returns an entire column of the grid.
- **T At(int r, int c):** returns the object at the listed indice after converting it from a Datum object.
- **T Find(T element):** Searches for an element.

Error conditions:

- 1) Programmer errors: Trying to access data in the grid outside of the capacity will throw an “Out of bounds” error.
- 2) Recoverable error: Memory may be a bottleneck in which case the class must be able to ensure proper allocation.
- 3) User error: The user will never be in direct contact with a DataGrid.

Serializer:

Class Description and Goals

The Serializer class converts C++ objects into string representations that can be stored in a database or sent over a network, and reconstructs those objects from their string form. The main goal is to provide a simple, consistent way to save and load game state without each class needing to implement its own file I/O logic. Under the hood, Serializer uses a text-based format with type tags and length prefixes so it knows how to reconstruct objects correctly. For simple types like int, double, and string, Serializer handles everything automatically. For custom classes made by other groups, those classes just need to provide a simple `Serialize()` method and the Serializer will call it.

Similar Standard Library Classes

`std::stringstream` `std::to_string` `std::stod` / `std::stoi` `std::variant` (used by Datum, which Serializer must support)

`std::optional` (for error handling on deserialization)

Key Functions:

The main functions are `Serialize()` overloads for each primitive type like `int`, `double`, `bool`, `char`, `std::string`. These just convert the value to a string with a type marker. For vectors and maps I'll encode the size first then loop through and serialize each element. Going the other way, there's `DeserializeInt()`, `DeserializeDouble()`, `DeserializeString()`, etc. that parse the string and give back the value. I also want a template version `Deserialize<T>()` where you specify what type you expect. `RegisterType()` lets other groups hook in their own classes. And obviously built-in support for `Datum` and `DataGrid` since we need those.

Error Conditions:

If someone tries to deserialize a malformed string that doesn't match the expected format, the function returns `std::nullopt` instead of crashing. Same deal if the type tag doesn't match what you asked for — like calling `DeserializeInt()` on something that was actually a string.

Programmer can just check for `nullopt` and handle it however they want. For stuff like passing `nullptr` or trying to serialize a custom type that was never registered, those hit an assert because that's just a bug in your code. If a number overflows during deserialization I'll probably throw an exception since that could actually happen legitimately with bad data.

Expected Challenges

Biggest issue is probably going to be strings that have my delimiter characters in them. Like if someone serializes the string "hello|world" and I'm using `|` as a delimiter, that breaks everything. So I need some kind of escape sequence system. Also doubles are annoying because `std::to_string()` rounds them weirdly, so I have to mess with `std::setprecision()` to not lose data. I haven't done much with C++ templates beyond basic stuff so the generic `Deserialize<T>()` might take some figuring out. And I need to test with Emscripten at some point to make sure it actually works in the browser.

Other groups' useful C++ classes:

Within our group, `Datum` and `DataGrid` since the point is to serialize those. `StringCompressor` will probably get chained with `Serializer` output to shrink stuff before storing it. `StringDiff` might use serialized strings to figure out what changed between updates. From other groups, `StateGrid` and `DataMap` (Group 7) hold world state that needs saving. `BehaviorTree` (Group 6) is complicated and nested so that'll be a good test case. `ActionLog` and `DataLog` (Group 23) need serialization for their replay feature. `FeatureVector` (Group 5) is basically just an array of numbers so that should be straightforward.

StringCompressor:

Class Description and Goals

Goals: Reducing the memory footprint of string data before it is stored in the `Datagrid`, or sent across the network.

The class will provide an interface to transform a `std::string` into a packed binary format and back again. It should prioritize execution speed over maximum compression ratio to ensure it does not bottle-neck the frame rate when running in a web browser.

Similar Standard Library Classes

`std::string/std::string_view`: For efficient handling of input data without copying when not needed.

`std::vector<uint8_t>` or `std::vector<char>`: To hold the raw compressed binary data

`std::bitset`: If implementing a bit-level compression

Key Functions:

// Compresses a standard string into a vector of bytes

`std::vector<char> Compress(const std::string& input);`

// Decompresses a vector of bytes back into the original string

`std::string Decompress(const std::vector<char>& compressed_data);`

// Optional: Returns the compression ratio achieved for the last operation

`double GetCompressionRatio();`

// Helper: Checks if the data is currently compressed (useful for the Serializer)

`bool IsCompressed(const std::vector<char>& data);`

Error Conditions and Responses

Error Condition	Source Type
Empty Input: Attempting to compress a string with a length of zero.	(1) Programmer Error (Should be caught by an assert)
Corrupted Header: The data provided for decompression does not begin with the expected class-specific signature.	(3) User Error (Returns a special condition/status code)
Checksum Mismatch: The decompressed data does not match the stored integrity check, indicating data corruption in the database.	(3) User Error (Returns a special condition/status code)

Memory Allocation Failure: The system is unable to allocate a large enough buffer for the decompressed output.	(2) Recoverable Error (Returns a special condition/status code)
Version Incompatibility: The data was compressed using a newer or incompatible version of the compression algorithm.	(3) User Error (Returns a special condition/status code)

Expected Challenges and Extra Topics

Algorithm Selection for Social Data: I need to research which compression algorithms perform best on the type of short, repetitive strings typical of social media "posts" or "messages" compared to large serialized objects.

Zero-Exception Architecture: Implementing robust error checking without the safety net of try-catch blocks. Probably will use `std::expected`.

Header Overhead: A major challenge is ensuring the "metadata" (like the original string length) doesn't take up more space than the compression saves, especially for very short strings.

Coordination with Other Groups

Group 23 (Data Analytics - ActionLog & DataLog): These classes track moves and values over time. I will coordinate with them to see if they want to pass their logs through the compressor before storage to prevent memory bloat during long simulation runs.

Group 24 (Web Interface - WebImage/WebCanvas): Since they are handling asset loading, I should check if they need a way to decompress string-encoded asset data arriving from the database before it is rendered to the UI.

Groups 7 & 8 (Worlds - DataMap): Both world groups use DataMap to store properties like "Hardness" or "Task Descriptions." I will work with them to ensure my compressor can handle the string values stored within their dynamic world maps.

StringDiff:

Description & Goals :

StringDiff computes and applies compact differences between two strings so incremental changes can be stored or transmitted instead of full copies. Its main goal is to provide a fast, lightweight way to update serialized data, logs, or state snapshots without expensive diff algorithms or heavy dependencies. Internally, StringDiff uses simple heuristics (such as common prefixes and suffixes) to produce “good enough” patches, and includes a lightweight signature to ensure patches are only applied to the correct base string. Patches can be encoded into a string-safe format for storage and later decoded and applied to reconstruct the updated string safely.

Similar Standard Library Classes

- `Std::string` / `std::string_view` - For string handling
- `Std::vector` - For storing diff operations
- `Std::pair` / `std::tuple` - For operation representation
- `Std::optional` - For reporting diff/apply failures without exceptions

Key Functions:

//Produces a path from base to updated

`Diff MakeDiff(const std::string& base, const std::string& updated);`

//Rebuilds the updated string from base and the patch. Returns empty if can't patch
`std::optional<std::string> ApplyDiff(const std::string& base, const Diff& patch);`

//Converts a path into a compact string format so it can be stored inside **Datum** / **DataGrid**

`Std::string EncodeDiff(const Diff& patch);`

//Restores a patch from stored from

`std::optional<Diff> DecodeDiff(const std::string& encoded);\\`

Error Conditions:

Programmer error (1) -> Assert:

- Passing an invalid patch object created by hand
- Calling Encode/Decode with null pointers.

Recoverable error (2) -> exception or `std::expected`:

- Patch grows beyond a configured maximum size

- Memory allocation failure while constructing the patch or result string

User/Data error (3) -> Special condition:

- Applying a patch to the wrong base string
 - Fix: Patch includes a lightweight signature
 - If sig doesn't match, ApplyDiff returns std::nullopt
- Decoding a wrong encoded diff data
 - DecodeDiff returns std::nullopt

Expected Challenges:

Algorithm Selection: A full diff is expensive. Want something fast and good enough. Can do things like (longest common prefix + longest common suffix), (treat middle portion as “changed block” for example

Proper encoding: If you store patches in strings, you must handle delimiter collisions cleanly. Solutions: length-prefix fields or something like base-64 encoding

Correctness for weird cases: Empty strings, full replacement, large insertions, unicode bytes
Emscripten constraints: Avoid heavyweight libraries. Keep memory allocations normal.

Coordination with Other Groups

- **Group 23: ActionLog, DataLog, ReplayDriver:** Could store incremental updates to logs or world snapshots more efficiently
- **Groups 7 & 8: DataMap, StateGrid:** World state updates might be stored as serialized strings, diffed per tick or per event
- **Group 23:** If the web client incremental UI updates, sending patches instead of full payloads can improve performance.

Vision for Main Module:

Our vision for the Database Module is to provide a high-performance database that serves as the backbone for the company's social interaction software. By integrating the flexible storage of the **DataGrid** with the lightweight adaptability of **Datum**, we aim to provide a storage system that is as dynamic as the worlds being built by Group 7 and 8. Recognizing that real-time social interaction requires massive data throughput. The **Serializer** will translate complex objects into storable formats, while the **StringCompressor** and **StringDiff** will work in tandem to minimize

the memory footprint and network latency. Our goal is to make data management invisible to the other development groups, providing them with a reliable API that allows agents to remember every interaction and worlds to persist every change without compromising the simulation's performance in the browser.